Ciclos while

Algoritmos y Programación

Prof. D.Sc. BARSEKH-ONJI Aboud

Facultad de Ingeniería
Universidad Anáhuac México

March 23, 2026

Contenido general

1. Introducción a los ciclos
2. Sintaxis y estructura del `while`
3. `while` con condición
4. `while True` y control con `break`
5. Comparación y buenas prácticas
6. Tareas

Agenda

1. Introducción a los ciclos
2. Sintaxis y estructura del `while`
3. `while` con condición
4. `while True` y control con `break`
5. Comparación y buenas prácticas
6. Tareas

¿Por qué necesitamos ciclos?

Motivación

En programación, muchas tareas requieren **repetir** un conjunto de instrucciones varias veces. Sin ciclos, tendríamos que escribir el mismo código una y otra vez.

Ejemplos de la vida real:

- Revisar todos los correos de una bandeja de entrada.
- Calcular la suma de 1000 números.
- Pedir al usuario que ingrese datos hasta que sean válidos.
- Actualizar una simulación en cada paso de tiempo.

Tipos de ciclos

Ciclo `for`

- Se usa cuando se conoce **de antemano** el número de iteraciones.
- Ejemplo: recorrer una lista, iterar *n* veces.

Ejemplo conceptual

"Ejecutar estas instrucciones **10 veces**"

Ciclo `while`

- Se usa cuando **no se sabe** cuántas veces se repetirá.
- Repite mientras una **condición sea verdadera**.

Ejemplo conceptual

"Ejecutar estas instrucciones **mientras** el usuario no diga 'salir' "

Agenda

1 Introducción a los ciclos

2 Sintaxis y estructura del `while`

3 `while` con condición

4 `while True` y control con `break`

5 Comparación y buenas prácticas

6 Tareas

Estructura del ciclo `while`

Definición

Un ciclo `while` ejecuta un bloque de código **repetidamente** mientras que una **condición lógica** sea `True`. Cuando la condición se vuelve `False`, el ciclo termina.

Sintaxis general en Python:

$$\underbrace{\texttt{while}}_{\text{palabra clave}} \quad \underbrace{\texttt{condición}}_{\text{expresión booleana}} \quad :$$

- La **condición** se evalúa *antes* de cada iteración.
- Si la condición es `False` desde el principio, el cuerpo **nunca** se ejecuta.
- El cuerpo del ciclo debe **modificar la condición** para evitar ciclos infinitos.

Diagrama de flujo del `while`

Flujo de ejecución:

1. Se evalúa la **condición**.
2. Si es `True` → se ejecuta el **cuerpo**.
3. Se regresa al paso 1.
4. Si es `False` → el ciclo **termina**.

Regla de oro

Siempre asegúrate de que la condición eventualmente se vuelva `False`, o usa `break` para salir del ciclo.

Agenda

1 Introducción a los ciclos

2 Sintaxis y estructura del `while`

3 `while` con condición

4 `while True` y control con `break`

5 Comparación y buenas prácticas

6 Tareas

while con condición explícita

Descripción

La forma más común. El ciclo repite **mientras la condición sea verdadera**. Normalmente se usa un **contador** o una variable que cambia en cada iteración.

Ejemplo 1 — Contador ascendente:

```
contador = 1            # Inicializacion

while contador <= 5:    # Condicion
    print(f"Iteracion:{contador}")
    contador += 1       # Actualizacion (ESENCIAL)

print("Ciclo terminado.")
```

while con condición explícita

Salida esperada

```
Iteracion numero: 1 Iteracion numero: 2 ... Iteracion numero:
5 Ciclo terminado.
```

Ejemplo 2 — Suma acumulada

Problema: Sumar todos los números del 1 al 100 usando `while`.

```python
n = 1
suma = 0

while n <= 100:
    suma += n    # suma = suma + n
    n += 1

print(f"La suma de 1 a 100 es: {suma}")
# Resultado: La suma de 1 a 100 es: 5050
```

Variables clave:

- `n`: contador (1 a 101)
- `suma`: acumulador

Ejemplo 3 — Validación de entrada

Problema: Pedir al usuario un número entre 1 y 10 hasta que lo ingrese correctamente.

```
numero = int(input("Ingresa un numero entre 1 y 10: "))

while numero < 1 or numero > 10:
    print("Numero invalido. Intenta de nuevo.")
    numero = int(input("Ingresa un numero entre 1 y 10: "))

print(f"Numero aceptado: {numero}")
```

Patrón: ciclo de validación

Este patrón es muy común en aplicaciones reales. El ciclo actúa como un **guardabarrera**: solo deja pasar datos correctos.

Ejemplo 4 — Adivina el número

Problema: El programa elige el número secreto 7; el usuario adivina hasta acertar.

```
secreto = 7
intento = int(input("Adivina el numero (1-10): "))
intentos = 1

while intento != secreto:
    if intento < secreto:
        print("Muy bajo. Intenta de nuevo.")
    else:
        print("Muy alto. Intenta de nuevo.")
    intento = int(input("Tu intento: "))
    intentos += 1

print(f"Correcto! Lo lograste en {intentos} intento(s).")
```

Ejemplo 4 — Adivina el número

Concepto reforzado

La condición `intento != secreto` se evalúa cada vez. El ciclo termina exactamente cuando el usuario acierta.

Agenda

1. Introducción a los ciclos
2. Sintaxis y estructura del `while`
3. `while` con condición
4. `while True` y control con `break`
5. Comparación y buenas prácticas
6. Tareas

while True: ciclo infinito controlado

Concepto

`while True` crea un ciclo **infinito intencionalmente**. La salida se controla con la instrucción `break`, que interrumpe el ciclo desde cualquier punto dentro del cuerpo.

¿Cuándo usarlo?

- Menús interactivos.
- Servidores o daemons.
- Cuando la condición de salida se calcula *dentro* del cuerpo.
- Re-intentos hasta éxito.

Diferencia clave

`while condición`**:** la condición se verifica *al inicio*.

`while True + break`**:** la condición de salida puede estar *en cualquier lugar* del cuerpo.

break y continue

break — salir del ciclo:

```python
i = 0
while True:
    if i == 3:
        break  # Sale del ciclo
    print(i)
    i += 1
# Imprime: 0, 1, 2
```

continue — saltar iteración:

```python
i = 0
while i < 6:
    i += 1
    if i == 4:
        continue # Salta al siguiente
    print(i)
# Imprime: 1, 2, 3, 5, 6
```

Resumen

- `break`: termina el ciclo inmediatamente.
- `continue`: omite el resto del cuerpo en la iteración actual y pasa a la siguiente.

Ejemplo 5 — Menú interactivo con while True

Problema: Menú de opciones que se repite hasta que el usuario elige salir.

```
while True:
    print("\n--- MENU ---")
    print("1. Saludar")
    print("2. Mostrar la fecha")
    print("3. Salir")

    opcion = input("Elige una opcion: ")

    if opcion == "1":
        print("Hola, bienvenido!")
    elif opcion == "2":
        print("Hoy es lunes.")
    elif opcion == "3":
        print("Hasta luego.")
        break           # Salida controlada
    else:
        print("Opcion invalida.")
```

Ejemplo 6 — Reintentos con while True

Problema: Solicitar contraseña hasta 3 intentos fallidos o éxito.

```
clave_correcta = "anahuac2025"
intentos = 0
MAX_INTENTOS = 3
while True:
    clave = input("Ingresa tu contrasena: ")
    intentos += 1

    if clave == clave_correcta:
        print("Acceso concedido.")
        break
    elif intentos >= MAX_INTENTOS:
        print("Cuenta bloqueada. Demasiados intentos.")
        break
    else:
        restantes = MAX_INTENTOS - intentos
        print(f"Clave incorrecta. Te quedan {restantes} intento(s).")
```

Ejemplo 6 — Reintentos con `while True`

Concepto reforzado

Dos condiciones de salida posibles dentro del cuerpo: éxito o límite de intentos.

Agenda

1. Introducción a los ciclos
2. Sintaxis y estructura del `while`
3. `while` con condición
4. `while True` y control con `break`
5. Comparación y buenas prácticas
6. Tareas

Comparación: while cond vs while True

Aspecto	`while condición`	`while True + break`
Condición de salida	Al *inicio* del ciclo	Dentro del cuerpo
Legibilidad	Alta (simple)	Media (requiere `break`)
Uso típico	Contadores, rangos	Menús, servidores, reintentos
Riesgo ciclo infinito	Bajo (con cuidado)	Alto sin `break`

Recomendación general

Prefiere `while condición` cuando sea posible. Usa `while True` solo cuando la condición de salida no puede evaluarse antes del cuerpo.

Errores comunes y cómo evitarlos

1. **Olvidar actualizar la variable de control**
   - Causa un **ciclo infinito**.
   - Solución: asegúrate de que `contador += 1` (o equivalente) esté en el cuerpo.
2. **Condición que nunca es `True` desde el inicio**
   - El cuerpo **nunca** se ejecuta.
   - Revisa los valores iniciales antes de entrar al ciclo.
3. **Error de indentación**
   - Python usa la **indentación** para delimitar el cuerpo del ciclo.
   - Cada línea del cuerpo debe tener exactamente 4 espacios de sangría.
4. **Usar `while True` sin `break`**
   - Programa colgado. Usa `Ctrl+C` para detenerlo.

Agenda

1. Introducción a los ciclos
2. Sintaxis y estructura del `while`
3. `while` con condición
4. `while True` y control con `break`
5. Comparación y buenas prácticas
6. Tareas

Tarea 1 — Tabla de multiplicar interactiva

Objetivo

Aplicar `while` con condición para generar tablas de multiplicar y practicar la validación de entrada del usuario.

Instrucciones:

1. Pide al usuario que ingrese un número entero entre 1 y 12. **Valida** la entrada: si está fuera del rango, vuelve a pedirlo (usa `while`).
2. Una vez validado, imprime la tabla de multiplicar completa del 1 al 12 usando un ciclo `while`.
3. Al terminar, pregunta: "*¿Deseas calcular otra tabla? (s/n)*". Si el usuario responde `s`, repite el proceso. Si responde `n`, termina.

Tarea 1 — Tabla de multiplicar interactiva

Ejemplo de salida esperada (número = 7)

```
7 x 1 = 7   7 x 2 = 14   ...   7 x 12 = 84
```

Criterios de evaluación

Validación de entrada (30%)	Ciclo de tabla (40%)	Repetición del programa (30%)

Tarea 2 — Adivina el número (versión completa)

Objetivo

Combinar `while True` con `break` y el módulo `random` para construir un juego interactivo completo.

Instrucciones:

1. Genera un número secreto **aleatorio** entre 1 y 50 usando `random.randint(1, 50)`.
2. Usa `while True` para pedir al usuario que adivine el número.
3. En cada intento, indica si el número es "demasiado alto", "demasiado bajo" o "correcto".
4. Cuando el usuario adivine, muestra el **número de intentos** utilizados y termina con `break`.
5. **Mejora opcional (+20% extra):** Agrega un límite de 7 intentos. Si se agota, revela el número secreto y termina.

Tarea 2 — Adivina el número (versión completa)

Criterios de evaluación

Generación aleatoria (20%) Lógica de pistas (40%) Conteo de intentos (20%) `break` correcto (20%)

Resumen

`while condición`

- Condición al inicio.
- Ideal para contadores y rangos.
- Requiere actualizar la variable de control.

`while True + break`

- Salida desde dentro del cuerpo.
- Ideal para menús y reintentos.
- Siempre debe haber un `break`.

Comandos adicionales

- `break`: salir del ciclo.
- `continue`: saltar iteración actual.

Recuerda

Todo ciclo `while` debe tener una condición de salida garantizada.